PS-049 — Project Design Summary

T072: Returns and Reverse Logistics Grading Engine

1. Project Overview

The Returns and Reverse Logistics Grading Engine is a Python-based system designed to process product return requests, validate return eligibility, manage return case states, evaluate returned product conditions, calculate refunds and recovery values, and generate reports.

The implementation supports the original JSON-based workflow and the official CSV-based teacher-pack workflow.

2. System Architecture

The project is organised into the following main components:

• `command_parser.py`: Parses and interprets commands from the supplied command input file.

• `data_loader.py`: Loads and prepares the official CSV input datasets.

• `dataset.py`: Provides dataset-related structures and supporting functionality.

• `official_engine.py`: Processes return cases, validates business rules, manages case states, and performs calculations using the official teacher-pack data.

• `official_cli.py`: Provides the command-line entry point for running the official sample processor.

• `output_writer.py`: Generates the required output CSV reports.

The tests directory contains automated tests for the baseline engine and official teacher-pack processing.

3. Input Data

The official sample processor uses the files provided in sample_data/teacher_pack/.

The input files include:

• `orders.csv`

• `customers.csv`

• `products.csv`

• `categories.csv`

• `sellers.csv`

• `accessories.csv`

• `grading.csv`

• `holidays.csv`

• `prior_returns.csv`

• `warehouse_receipts.csv`

• `commands.txt`

The earlier sample_data/commands.json file is retained for the baseline JSON workflow.

4. Main Processing Workflow

1. Load the required order, customer, product, category, seller, grading and other CSV datasets.
2. Read and parse the supplied return-processing commands.
3. Validate order details, timestamps, return eligibility and business rules.
4. Create and manage return cases.
5. Process pickup attempts and warehouse receipt information.
6. Verify returned product serial numbers and inspection details.
7. Calculate the product condition score and assign the appropriate grade.
8. Determine the product's recovery route.
9. Calculate refund amounts, deductions, recovery values, courier claims and platform loss.
10. Generate case files, state logs and weekly reports.

5. Product Grading and Recovery

The system evaluates returned products based on inspection information.

The grading process considers scratches, functional faults, missing boxes, broken seals and product usage.

Products are assigned one of four grades:

- Grade A: Suitable for restocking.
- Grade B: Sent for refurbishment, subject to quality-control results.
- Grade C: Liquidated.
- Grade D: Scrapped.

A Grade B product that fails quality control is routed for liquidation according to the implemented business rules.

6. Special Business Scenarios

The system handles important return-processing scenarios:

- Ineligible return requests are rejected.
- Serial-number mismatches are flagged for fraud review.
- Failed pickup attempts are tracked.
- Eligible cases can be reopened within the configured time limit.
- Grade C and Grade D refunds require manager approval.
- Exchange cases are processed separately from cash refunds.
- Accessory deductions and courier claims are subject to configured limits.

• Invalid commands are recorded as command errors instead of being processed as successful operations.

7. State Management

The return-case lifecycle is controlled through defined states and validated transitions.

The system checks whether a requested state transition is permitted before changing the case state. Illegal transitions raise errors and prevent invalid workflow progression.

Reopening and other special workflow operations are handled using their respective validation rules.

8. Output Generation

The official sample processor generates the following CSV output files in the output/ directory:

• `case_files.csv`: Contains the processed return-case details and financial calculations.
• `state_log.csv`: Records the state transitions and processing history.
• `weekly_report.csv`: Provides weekly return-processing and reporting information.

Invalid commands are also tracked during processing.

9. Testing and Validation

The project uses Python's built-in unittest framework.

The test suite contains:

• `test_engine.py`: Tests the baseline engine and business rules.
• `test_official_pack.py`: Tests official teacher-pack processing and important sample scenarios.

A total of 34 automated tests passed during the latest local test run.

The tests cover grading boundaries, refund calculations, return eligibility, serial mismatches, pickup failures, exchange handling, recovery calculations, invalid commands and other business scenarios.

10. Conclusion

The project provides a structured Python implementation for processing product returns and reverse logistics operations. It combines input validation, return eligibility checking, state management, product grading, financial calculations, automated testing and CSV report generation.

The implementation has been extended from the initial Week 2 baseline to support the supplied official teacher-pack datasets and command-processing workflow. Remaining provisional business assumptions are documented in docs/DECISION_LOG.md.